- External JavaScript provides **code reusability**: one .js file can be used by many HTML pages.
- An external JavaScript file must be saved with the **.js** extension.

4. Variables, Data Types and Strings

A **variable** is a named container used to store a value. In JavaScript, a variable can hold different types of values.

Common data types

Data type	Meaning	Example
String	**Text wrapped in single or double quotes**	**"rajkot"**
Number	**Numeric value, integer or decimal**	**156, 100, 1.2**
Boolean	**True or false value**	**true, false**
Null	**Intentionally empty value**	**null**
Function	**A function name or function value**	**myFunction**
Object	**Collection of related properties and methods**	**{name:"Darshan"}**

Rules for JavaScript variable names

- Variable names are case-sensitive: **mynum** and **MyNum** are different.
- A variable name cannot contain punctuation or spaces.
- A variable name cannot start with a digit.
- A variable name cannot be a JavaScript reserved word, such as **if**, **for**, or **function**.

Strings

A **string** is a sequence of letters, digits, punctuation marks, and other characters. Strings are written inside single or double quotes.

```
var mystring = "my college name is " + "Darshan";
var lenStr = mystring.length;   // returns string length as an integer
```

The **+** operator joins strings; this is called **concatenation**. The backslash **** is an escape character used to include special characters such as quotes inside a string.

```
var text = "I am studying in \"Darshan\" University.";
```

Important string functions

Function	Purpose	Example / result
concat()	Joins two or more strings	"Darshan".concat(" University")
length	Returns number of characters	"Darshan".length → 7
charAt()	Returns character at a given index	"Darshan".charAt(0) → "D"
indexOf()	Returns first position of a value	Finds position of a substring
lastIndexOf()	Returns last position of a value	Searches from the end
includes()	Checks whether a string contains a value	Returns true/false
startsWith() / endsWith()	Checks beginning or ending text	Returns true/false
replace()	Replaces matched text	"aa".replace("a","b") → "ba"
split()	Splits a string into an array	"a,b".split(",") → ["a","b"]

Function	Purpose	Example / result
slice(start,end)	Extracts part of a string	"Darshan".slice(0,3) → "Dar"
substring(start,end)	Extracts part of a string	Similar to slice
substr(start,length)	Extracts part using start and length	Extracts fixed number of characters
toLowerCase()	Converts to small letters	"DARSHAN".toLowerCase() → "darshan"
toUpperCase()	Converts to capital letters	"darshan".toUpperCase() → "DARSHAN"
trim()	Removes spaces from both ends	" hi ".trim() → "hi"
repeat()	Repeats a string	"ab".repeat(2) → "abab"
toString()	Converts a value to a string	(10).toString() → "10"
match() / search()	Pattern matching using regular expressions	Used with RegExp

5. Numbers and Arithmetic Operators

JavaScript numbers can be integers or decimals. JavaScript also provides functions to convert values into numbers and format them.

Important number functions

Function	Purpose	Example
Number()	**Converts a value to a number**	**Number("10") → 10; Number("John") → NaN**
parseInt()	**Converts to an integer**	**parseInt("10.33") → 10**
parseFloat()	**Converts to a decimal number**	**parseFloat("10.33") → 10.33**
toFixed()	**Formats a number to fixed decimal places**	**(9.656).toFixed(2) → "9.66"**
toPrecision()	**Formats a number to a given length**	**Used for precision control**
toExponential()	**Converts a number to exponential notation**	**Scientific notation**
valueOf()	**Returns the primitive value of a number**	**Returns numeric value**

NaN means “Not a Number”. It appears when a value cannot be converted into a valid number, for example **parseInt("years 10")**.

Arithmetic operators

Operator	Name	Example	Result
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
%	Modulus / remainder	5 % 2	1
++	Increment	x++	Increases x by 1
--	Decrement	x--	Decreases x by 1
**	Exponentiation	5 ** 2	25

The ****** operator was added in ES2016.

6. Arrays

An **array** is a collection of multiple values stored under one variable name. Each value is called an element and has an index. Array indexes start from **0**.

Ways to create arrays

```
// Method 1: create empty array and assign values
var myArray = new Array();
myArray[0] = "darshan";
myArray[1] = 222;
myArray[2] = false;

// Method 2: create array with values
var myArray = new Array("darshan", 123, true);

// Method 3: array literal (common modern method)
var myArray = ["darshan", 123, true];
```

In the array **["darshan", 123, true]**, index 0 is "darshan", index 1 is 123, and index 2 is true. JavaScript arrays can store different data types in the same array.

Array example

```
var num = [4, 6, 9, 8, 2];
var sum = 0;

for (let x of num) {
  sum += x;
}
alert(sum);   // Output: 29
```

7. Functions

A **function** is a block of reusable code designed to perform a particular task. A function runs only when it is called or invoked.

Function syntax

```
function functionName(parameter1, parameter2) {
  // code to be executed
  return value;
}
```

Key points

- The **function** keyword declares a function.
- The function name is followed by parentheses **()**.
- Parameters are inputs written inside the parentheses and separated by commas.
- The function body is written inside curly brackets **{ }**.
- The **return** statement sends a result back to the code that called the function.
- When JavaScript reaches **return**, the function stops executing.

Example

```
function myFunction(p1, p2) {
  return p1 * p2;
}

var result = myFunction(4, 3);
alert(result);   // Output: 12
```

Ways a function can be invoked

- When an event occurs, such as a button click.
- When it is called from another JavaScript statement.
- Automatically, as a self-invoking function.

8. Conditional Statements

Conditional statements allow a program to make decisions. They run different code depending on whether a condition is true or false.

Statement	Use
if	**Runs code only when a condition is true**
if...else	**Runs one block for true and another for false**
else if ladder	**Checks several conditions one by one**
Nested if	**An if statement inside another if statement**
switch	**Checks one value against many possible cases**

Simple if

```
var n = -25;
if (n < 0) {
  alert("Number is negative");
}
```

if...else

```
var n = 25;
if (n % 2 == 0) {
  alert("Number is even");
} else {
  alert("Number is odd");
}
```

else if ladder

```
var x = 10, y = 20;
var op = '+';

if (op == '+') {
  alert("Answer: " + (x + y));
} else if (op == '-') {
  alert("Answer: " + (x - y));
} else if (op == '*') {
  alert("Answer: " + (x * y));
} else if (op == '/') {
  alert("Answer: " + (x / y));
} else {
  alert("Invalid operator");
}
```

Nested if

```
var x = 10, y = 20, z = 15;

if (x > y) {
  if (x > z) {
    alert("Max = " + x);
  } else {
    alert("Max = " + z);
  }
} else {
  if (y > z) {
    alert("Max = " + y);
  } else {
    alert("Max = " + z);
  }
}
```

switch statement

A **switch** statement compares one value with several possible cases. The **break** statement stops the switch after a matching case runs. The **default** case runs when no case matches.

```
var n = parseInt(prompt("Enter Day Number"));

switch (n) {
  case 1: alert("Monday"); break;
  case 2: alert("Tuesday"); break;
  case 3: alert("Wednesday"); break;
  case 4: alert("Thursday"); break;
  case 5: alert("Friday"); break;
  case 6: alert("Saturday"); break;
  case 7: alert("Sunday"); break;
  default: alert("Wrong Input"); break;
}
```

Common mistake: forgetting **break** in a switch statement. Without break, execution continues into the next case; this is called fall-through.

9. Loops

A **loop** repeats a block of code several times. JavaScript supports while, do...while, for, for...in, and for...of loops.

Loop	Best used for
while	**Repeating while a condition remains true; condition is checked first**
do...while	**Running the block at least once; condition is checked after execution**
for	**Repeating a known number of times**
for...in	**Looping through object properties or array indexes**
for...of	**Looping through values of arrays, strings, and other iterable objects**

while loop

```
var i = 1;
while (i <= 10) {
  console.log(i);
  i++;
}
// Output: 1 2 3 4 5 6 7 8 9 10
```

do...while loop

```
var i = 1;
do {
  console.log(i);
  i++;
} while (i <= 10);
// Output: 1 2 3 4 5 6 7 8 9 10
```

for loop

```
for (var i = 1; i <= 10; i++) {
  console.log(i);
}
// Output: 1 2 3 4 5 6 7 8 9 10
```

for...in loop

The **for...in** loop gives the keys or indexes. It can be used with objects and arrays, but it should not be used with arrays when index order is important.

```
var num = [4, 6, 9, 8, 2];
let sum = 0;
for (let x in num) {
  sum += num[x];
}
alert(sum);   // 29

var person = {fname:"Dharmik", mname:"G.", lname:"Aghera"};
var name = "";
for (var x in person) {
  name += person[x] + " ";
}
alert(name);  // Dharmik G. Aghera
```

for...of loop

The **for...of** loop gives the actual values of an iterable object such as an array or string.

```
var num = [4, 6, 9, 8, 2];
let sum = 0;
for (let x of num) {
  sum += x;
}
alert(sum);   // 29

var s = "Darshan";
for (let x of s) {
  alert(x);   // Displays each character separately
}
```

- **break**: immediately exits the entire loop.
- **continue**: skips the current iteration and moves to the next one.

10. Pop-up Boxes

JavaScript provides three types of pop-up boxes: alert, confirm, and prompt.

Pop-up box	Purpose	Buttons	Return value
alert()	Shows information to the user	OK	No meaningful return value
confirm()	Asks the user to accept or cancel an action	OK / Cancel	true for OK, false for Cancel
prompt()	Takes text input from the user	OK / Cancel	Entered value for OK; null for Cancel

Alert box

```
alert("Hello World");
```

Confirm box

```
var a = confirm("Are you sure??");

if (a == true) {
  alert("User Accepted");
} else {
  alert("User Cancelled");
}
```

Prompt box

```
var a = prompt("Enter Name");
alert("User Entered " + a);
```

11. JavaScript Objects

An **object** is a special type of data that has **properties** and **methods**. Properties store values; methods are actions or functions belonging to the object.

Term	Meaning	Syntax / example
Property	**A value associated with an object**	**objectName.propertyName**
Method	**An action performed by an object**	**objectName.methodName()**
String object example	**Finds string length**	**message.length**
String method example	**Converts text to uppercase**	**message.toUpperCase()**

Built-in objects

- String
- Date
- Array
- Boolean
- Math
- RegExp

Math object properties

11. JavaScript Objects

An **object** is a special type of data that has **properties** and **methods**. Properties store values; methods are actions or functions belonging to the object.

Term	Meaning	Syntax / example
Property	**A value associated with an object**	**objectName.propertyName**
Method	**An action performed by an object**	**objectName.methodName()**
String object example	**Finds string length**	**message.length**
String method example	**Converts text to uppercase**	**message.toUpperCase()**

Built-in objects

- String
- Date
- Array
- Boolean
- Math
- RegExp

Math object properties

Property	Description
Math.E	Euler’s number, approximately 2.718
Math.LN2	Natural logarithm of 2, approximately 0.693
Math.LN10	Natural logarithm of 10, approximately 2.302
Math.LOG2E	Base-2 logarithm of E, approximately 1.442
Math.LOG10E	Base-10 logarithm of E, approximately 0.434
Math.PI	Value of pi, approximately 3.14
Math.SQRT1_2	Square root of 1/2
Math.SQRT2	Square root of 2

Important Math methods

Method	Purpose
Math.abs(x)	Absolute value of x
Math.sin(x), cos(x), tan(x)	Trigonometric values; x is in radians
Math.asin(x), acos(x), atan(x)	Inverse trigonometric values
Math.random()	Random number between 0 and 1
Math.ceil(x)	Rounds x upward to the nearest integer
Math.floor(x)	Rounds x downward to the nearest integer
Math.round(x)	Rounds x to the nearest integer
Math.pow(x,y)	Calculates x raised to the power y
Math.sqrt(x)	Square root of x
Math.max(x,y,...)	Largest value
Math.exp(x)	E raised to the power x
Math.log(x)	Natural logarithm of x

User-defined objects

JavaScript allows you to create your own objects. One way is to use the **new Object()** constructor; another common way is object literal notation.

```
var person = {
  firstname: "Darshan",
  lastname: "College",
  age: 50,
  eyecolor: "blue"
};

alert(person.firstname + " is " + person.age + " years old.");
```

A **constructor** is a function used with the **new** keyword to create and initialize objects. Inside a constructor, **this** refers to the current object being created.

```
function person(firstname, lastname, age) {
  this.firstname = firstname;
  this.lastname = lastname;
  this.changeFirstName = function(name) {
    this.firstname = name;
  };
```

```
}

var person1 = new person("Narendra", "Modi", 50);
person1.changeFirstName("NAMO");
alert(person1.firstname + " " + person1.lastname);
// Output: NAMO Modi
```

12. Built-in Functions

A **built-in function** is already available in JavaScript and can be used directly without defining it.

Function	Purpose
eval()	**Evaluates or executes a string as JavaScript code**
isFinite()	**Checks whether a value is a finite number**
isNaN()	**Checks whether a value is NaN (Not a Number)**
Number()	**Converts a value to a number**
parseInt()	**Converts a value to an integer**
parseFloat()	**Converts a value to a floating-point number**
String()	**Converts a value to a string**
console.log()	**Prints output in the browser console**

13. Form Validation and Regular Expressions

Validation is the process of checking data against a required standard or rule. JavaScript can validate form data in the browser before it is sent to the server, reducing server load and giving faster feedback.

Types of validation

Type	What it checks	Examples
Basic validation	**Whether required data is present and correct in a simple way**	**Empty field check, confirm password, length check**
Data format validation	**Whether data follows the correct format and value pattern**	**Email, mobile number, enrolment number**

Regular expressions

A **regular expression (RegExp)** is an object that describes a pattern of characters. It is used for pattern matching, searching, validation, and search-and-replace operations on text.

```
var pattern = "^[\\w]+$";       // allows only word characters
var regex = new RegExp(pattern);

if (regex.test(testString)) {
  // Valid
} else {
  // Invalid
}
```

RegExp modifiers

Modifier	Meaning
g	**Global search: finds all matches**
i	**Case-insensitive matching**
m	**Multiline matching**

RegExp brackets

Expression	Meaning
[abc]	**Matches any one character a, b, or c**
[^abc]	**Matches any character except a, b, or c**
[0-9]	**Matches any digit from 0 to 9**
[^0-9]	**Matches any character except digits**
(x|y)	**Matches either x or y**

Metacharacters

Symbol	Meaning
.	**Any single character except newline**
\w	**Word character: a-z, A-Z, 0-9, and underscore**
\W	**Non-word character**
\d	**Digit, same as [0-9]**

Symbol	Meaning
\D	Non-digit character
\s	Whitespace character
\S	Non-whitespace character
\t	Tab character
\n	New line
\0	NULL character

Quantifiers

Quantifier	Meaning
n+	One or more occurrences of n
n*	Zero or more occurrences of n
n?	Zero or one occurrence of n
^n	String must begin with n
n$	String must end with n
n{X}	Exactly X occurrences of n
n{X,Y}	Between X and Y occurrences of n
n{X,}	At least X occurrences of n

Name validation

```
var a = "Darshan-University";
var regex = /^[a-zA-Z0-9]+$/;

if (regex.test(a)) {
  alert("Valid");
} else {
  alert("Invalid");
}
// "Darshan-University" is invalid because "-" is not allowed.
```

- **/**: start and end delimiters of the regular expression.
- **^**: start of string.
- **[a-zA-Z0-9]**: one letter or digit.
- **+**: one or more occurrences.
- **$**: end of string.

Phone number validation

```
var a = "123-456-7890";
var regex = /^\d{3}-\d{3}-\d{4}$/;

if (regex.test(a)) {
  alert("Valid");
} else {
  alert("Invalid");
}
```

- **\d{3}**: exactly three digits.

- **-**: literal hyphen.
- **\d{4}**: exactly four digits.
- Valid example: 123-456-7890; invalid examples: 123-45-67890, 1234567890, 123-456-78a0.

Email validation

```
var a = "dharmikaghera@gmail.com";
var regex = /^([a-zA-Z0-9._%+-]+)@([a-zA-Z0-9.-]+\.[a-zA-Z]{2,})$/;

if (regex.test(a)) {
  alert("Valid");
} else {
  alert("Invalid");
}
```

- The first bracket matches the local part before @.
- The second bracket matches the domain after @.
- **\.** requires a dot before the top-level domain.
- **[a-zA-Z]{2,}** requires at least two letters in the domain ending, such as com, org, or io.

Form-based email validation

```
function checkMail() {
  var a = document.getElementById("myText").value;
  var pattern = "^[\\w-_.]*[\\w-_.]\\@[\\w]\\.+[\\w]+[\\w]$";
  var regex = new RegExp(pattern);

  if (regex.test(a)) {
    alert("Valid");
  } else {
    alert("Invalid");
  }
}
```

14. Callbacks and Functions as Arguments

A **callback function** is a function passed into another function as an argument. The outer function calls it later, usually after completing a task.

Callback example

```
function printResult(some) {
  document.getElementById("demo").innerHTML = some;
}

function myCalculator(num1, num2, myCallback) {
  let sum = num1 + num2;
  myCallback(sum);
}

myCalculator(5, 5, printResult);
// printResult receives 10 and displays it
```

- **myCalculator** receives two numbers and a callback function.
- It calculates the sum.
- It calls **myCallback(sum)**, which runs **printResult**.
- The callback displays the result.

Function as an argument

In JavaScript, functions can be passed as arguments to other functions. This makes code flexible because the same outer function can perform different operations.

```
function calculate_area(l, b) {
  return l * b;
}

function calculate_perimeter(l, b) {
  return 2 * (l + b);
}

function rectangle_calculator(l, b, property) {
  var ans = property(l, b);
  return ans;
}

console.log("Area: " + rectangle_calculator(2, 5, calculate_area) + " squareunit");
console.log("Perimeter: " + rectangle_calculator(2, 5, calculate_perimeter) + " unit");

// Output:
// Area: 10 squareunit
// Perimeter: 14 unit
```

15. Final Revision

Chapter-wise quick revision

Topic	Remember this
JavaScript	Client-side scripting language that makes pages dynamic and interactive
HTML / CSS / JavaScript	HTML = structure, CSS = styling, JavaScript = behaviour
Ways to add JS	Inline, internal/embedded, external (.js file)
Variable	Named container for a value; case-sensitive
String	Text in quotes; joined using + or concat()
Array	Indexed collection of values; index starts at 0
Function	Reusable block of code; runs when called
if / switch	Decision-making statements
Loops	while, do...while, for, for...in, for...of
break / continue	break exits loop; continue skips one iteration
Pop-ups	alert(), confirm(), prompt()
Object	Has properties and methods
Math object	Provides mathematical constants and methods
Validation	Checking data before sending it to the server
RegExp	Pattern used to validate or search text
Callback	Function passed as an argument and called later

Key definitions

- **Client-side scripting:** Code executed by the user's browser.
- **Variable:** A named storage location for a value.
- **String:** A sequence of characters enclosed in quotes.
- **Array:** An ordered collection of values accessed using indexes.
- **Function:** A named block of code that performs a task.
- **Callback:** A function passed to another function as an argument.
- **Validation:** Checking whether data meets required rules.
- **Regular expression:** A pattern used to match, search, or validate text.

Confusing concepts

Concepts	Difference
while vs do...while	while checks the condition first; do...while executes once before checking
for...in vs for...of	for...in gives indexes/keys; for...of gives values
break vs continue	break stops the whole loop; continue skips only the current iteration

Concepts	Difference
alert vs confirm vs prompt	**alert shows information; confirm asks yes/no; prompt takes input**
slice vs substring	**Both extract parts of strings; slice can accept negative indexes**
parseInt vs parseFloat	**parseInt returns an integer; parseFloat returns a decimal number**
client-side vs server-side	**Browser executes client-side; server executes server-side**

16. Practice Questions and Answer Key

Short-answer questions

- What is JavaScript and why is it called a client-side scripting language?
- List any four tasks performed by client-side scripts.
- Write the difference between inline, internal, and external JavaScript.
- What is a variable? State any three rules for naming JavaScript variables.
- What is string concatenation? Give an example.
- What is an array? Write its index starting value.
- What is a function in JavaScript?
- Differentiate between while and do...while loops.
- What is the difference between break and continue?
- Name the three JavaScript pop-up boxes and state their return values.
- What is form validation?
- What is a regular expression?
- What is a callback function?

Long-answer questions

- Explain client-side and server-side scripting with a comparison table.
- Explain the types of JavaScript with suitable HTML and JavaScript examples.
- Explain JavaScript conditional statements with examples.
- Explain all types of JavaScript loops with syntax and output.
- Explain alert, confirm, and prompt boxes with examples.
- Explain JavaScript objects, properties, methods, and constructors with examples.
- Explain regular expression modifiers, brackets, metacharacters, and quantifiers.
- Explain email validation using a regular expression with code.
- Explain callbacks and functions as arguments with examples.

MCQs

No.	Question	A	B	C	D	Answer
1	Which language is mainly used for client-side scripting?	HTML	CSS	JavaScript	SQL	c
2	Which attribute is used to link an external JavaScript file?	link	src	href	include	b
3	Array indexes in JavaScript start from:	-1	0	1	Depends on array	b
4	Which operator finds the remainder?	/	%	**	++	b
5	Which loop always executes at least once?	for	while	do...while	for...of	c
6	Which loop gives array values directly?	for...in	for...of	while	do...while	b
7	Which pop-up box returns true or false?	alert	prompt	confirm	console.log	c
8	Which pop-up box takes user input?	alert	confirm	prompt	none	c
9	What does NaN mean?	Not a Name	Not a Number	New and Null	No Available Number	b

No.	Question	A	B	C	D	Answer
10	Which RegExp symbol matches one or more occurrences?	*	+	?	$	b
11	Which RegExp symbol matches digits?	\w	\d	\s	\D	b
12	Which statement exits a loop completely?	continue	return	break	exit	c

15. Final Revision

Chapter-wise quick revision

Topic	Remember this
JavaScript	**Client-side scripting language that makes pages dynamic and interactive**
HTML / CSS / JavaScript	**HTML = structure, CSS = styling, JavaScript = behaviour**
Ways to add JS	**Inline, internal/embedded, external (.js file)**
Variable	**Named container for a value; case-sensitive**
String	**Text in quotes; joined using + or concat()**
Array	**Indexed collection of values; index starts at 0**
Function	**Reusable block of code; runs when called**
if / switch	**Decision-making statements**
Loops	**while, do...while, for, for...in, for...of**
break / continue	**break exits loop; continue skips one iteration**
Pop-ups	**alert(), confirm(), prompt()**
Object	**Has properties and methods**
Math object	**Provides mathematical constants and methods**
Validation	**Checking data before sending it to the server**
RegExp	**Pattern used to validate or search text**
Callback	**Function passed as an argument and called later**

Key definitions

- **Client-side scripting:** Code executed by the user's browser.
- **Variable:** A named storage location for a value.
- **String:** A sequence of characters enclosed in quotes.
- **Array:** An ordered collection of values accessed using indexes.
- **Function:** A named block of code that performs a task.
- **Callback:** A function passed to another function as an argument.
- **Validation:** Checking whether data meets required rules.
- **Regular expression:** A pattern used to match, search, or validate text.

Confusing concepts

Concepts	Difference
while vs do...while	**while checks the condition first; do...while executes once before checking**
for...in vs for...of	**for...in gives indexes/keys; for...of gives values**
break vs continue	**break stops the whole loop; continue skips only the current iteration**
alert vs confirm vs prompt	**alert shows information; confirm asks yes/no; prompt takes input**
slice vs substring	**Both extract parts of strings; slice can accept negative indexes**
parseInt vs parseFloat	**parseInt returns an integer; parseFloat returns a decimal number**
client-side vs server-side	**Browser executes client-side; server executes server-side**

16. Practice Questions and Answer Key

Short-answer questions

- What is JavaScript and why is it called a client-side scripting language?
- List any four tasks performed by client-side scripts.
- Write the difference between inline, internal, and external JavaScript.
- What is a variable? State any three rules for naming JavaScript variables.
- What is string concatenation? Give an example.
- What is an array? Write its index starting value.
- What is a function in JavaScript?
- Differentiate between while and do...while loops.
- What is the difference between break and continue?
- Name the three JavaScript pop-up boxes and state their return values.
- What is form validation?
- What is a regular expression?
- What is a callback function?

Long-answer questions

- Explain client-side and server-side scripting with a comparison table.
- Explain the types of JavaScript with suitable HTML and JavaScript examples.
- Explain JavaScript conditional statements with examples.
- Explain all types of JavaScript loops with syntax and output.
- Explain alert, confirm, and prompt boxes with examples.
- Explain JavaScript objects, properties, methods, and constructors with examples.
- Explain regular expression modifiers, brackets, metacharacters, and quantifiers.
- Explain email validation using a regular expression with code.
- Explain callbacks and functions as arguments with examples.

MCQs

No.	Question	A	B	C	D	Answer
1	Which language is mainly used for client-side scripting?	HTML	CSS	JavaScript	SQL	c
2	Which attribute is used to link an external JavaScript file?	link	src	href	include	b
3	Array indexes in JavaScript start from:	-1	0	1	Depends on array	b
4	Which operator finds the remainder?	/	%	**	++	b
5	Which loop always executes at least once?	for	while	do...while	for...of	c
6	Which loop gives array values directly?	for...in	for...of	while	do...while	b
7	Which pop-up box returns true or false?	alert	prompt	confirm	console.log	c
8	Which pop-up box takes user input?	alert	confirm	prompt	none	c
9	What does NaN mean?	Not a Name	Not a Number	New and Null	No Available Number	b

No.	Question	A	B	C	D	Answer
10	Which RegExp symbol matches one or more occurrences?	*	+	?	$	b
11	Which RegExp symbol matches digits?	\w	\d	\s	\D	b
12	Which statement exits a loop completely?	continue	return	break	exit	c

Answer key: 1-c, 2-b, 3-b, 4-b, 5-c, 6-b, 7-c, 8-c, 9-b, 10-b, 11-b, 12-c

Short answers

• JavaScript is a client-side scripting language because its code is executed by the web browser to make pages interactive.

• Client-side scripts validate data, handle events, update page parts, change styles dynamically, and generate HTML content.

• Inline JavaScript is written in an HTML element; internal JavaScript is written in <script> tags in the same page; external JavaScript is stored in a separate .js file.

• A variable is a named container for storing data. Its name is case-sensitive, cannot start with a digit, and cannot be a reserved word.

• String concatenation means joining strings, for example: "Darshan" + " University".

• An array is a collection of values stored under one name; indexes begin at 0.

• A function is a reusable block of code that performs a task when called.

• while checks the condition before execution; do...while executes once and then checks the condition.

• break exits the loop; continue skips the current iteration.

• alert() shows a message, confirm() returns true/false, and prompt() returns entered text or null.

• Validation checks whether entered data meets required rules before processing.

• A regular expression is a character pattern used for matching, searching, and validating text.

• A callback is a function passed as an argument to another function and invoked later.

Last-minute checklist

• Can you explain HTML, CSS, and JavaScript roles?

• Can you write internal and external JavaScript examples?

• Do you know variable naming rules and data types?

• Can you use string and number functions?

• Can you create arrays and calculate their sum?

• Can you write functions with parameters and return values?

• Can you write if, if...else, else if, nested if, and switch programs?

• Can you write all five loops and explain break/continue?

• Do you know the return values of alert, confirm, and prompt?

• Can you create objects using literals and constructors?

• Do you know common Math methods?

• Can you write RegExp for name, phone number, and email validation?

• Can you explain callbacks and passing functions as arguments?